МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод бисекции.

Студент гр. 4342 Кринкин Н.К.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 8. $f(x) = 2x^2 - x^4 - 1 + ln(x)$

Изучить метод бисекции для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Задание.

В лабораторной работе №3 предлагается, используя программы - функции BISECT и Round из файла methods.cpp (файл заголовков methods.h), найти корень уравнения $f(x) = 0$ методом бисекции с заданной точностью Eps, исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных). Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений),

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$

3) Составить головную программу, содержащую обращение к подпрограмме f(x), BISECT, Round и индикацию результатов.

4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.

5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

Основные теоретические положения.

Если найден отрезок [a,b], такой, что $f(a)f(b) < 0$, существует точка c, в которой значение функции равно нулю, т.е. $f(x) = 0,\ c \in (a, b)$. Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции $f(x)$ (корень уравнения $f(x) = 0$) с любой заданной точностью.

Рассмотрим один шаг итерационного процесса. Пусть на (n-1)-м шаге найден отрезок $[a_{n-1},\ b_{n-1}] \subset [a,\ b]$, такой, что $f(a_{n-1})f(b_{n-1}) < 0$. Разделим его пополам точкой $\varepsilon = (a_{n-1} + b_{n-1}) / 2$ и вычислим $f(\varepsilon)$. Если $f(\varepsilon) = 0$, то $\varepsilon = (a_{n-1} + b_{n-1}) / 2$ – корень уравнения. Если $f(\varepsilon) \neq 0$, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

$a_n = a_{n-1},\ b_n = \varepsilon$, если $f(\varepsilon)\, f(a_{n-1}) < 0$

$a_n = \varepsilon,\ b_n = b_{n-1}$, если $f(\varepsilon)\, f(a_{n-1}) > 0$

Если требуется найти корень с точностью ε, то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ε. Тогда координата середины отрезка есть значение корня с требуемой точностью ε.

Метод бисекции является простым и надежным методом поиска простого корня уравнения $f(x) = 0$ (простым называется корень x=c дифференцируемой функции $f(x)$, если $f(c)$ и $f'(c) \neq 0$). Этот метод сходится для любых непрерывных функций $f(x)$, в том числе недифференцируемых. Скорость его сходимости невысока. Для достижения точности ε необходимо совершить $N \approx log_2(b - a) / \varepsilon$ итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

Выполнение работы.

1) Нахождение отрезка

С помощью бесплатного графического онлайн калькулятора построим график функции $f(x) = 2x^2 - x^4 - 1 + ln(x)$. Полученный график представлен на рисунке 1.

Рисунок 1 - график функции $f(x) = 2x^2 - x^4 - 1 + ln(x)$

По графику видно, что для вычисления можно подобрать отрезок (a, b), где $a \in (0.5,\ 1)$ и $b \in (1,\ 1.5)$. В рамках данной работы возьмем отрезок [0.6432; 1.4432341923].

2) Реализуем функцию f(x), используя инструменты стандартной библиотеки C++:

```
double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}
```

Полный код программы представлен в приложении А.

3) В основной файле определим функцию main, которая представляет, которая представляет из себя цикл, перебирающий значение порядка точности ε и выводит результат работы функции BISECT с точностью delta (в данной работе – delta = 10^-4).

4) Проведены вычисления, построен график (Рисунок 2).

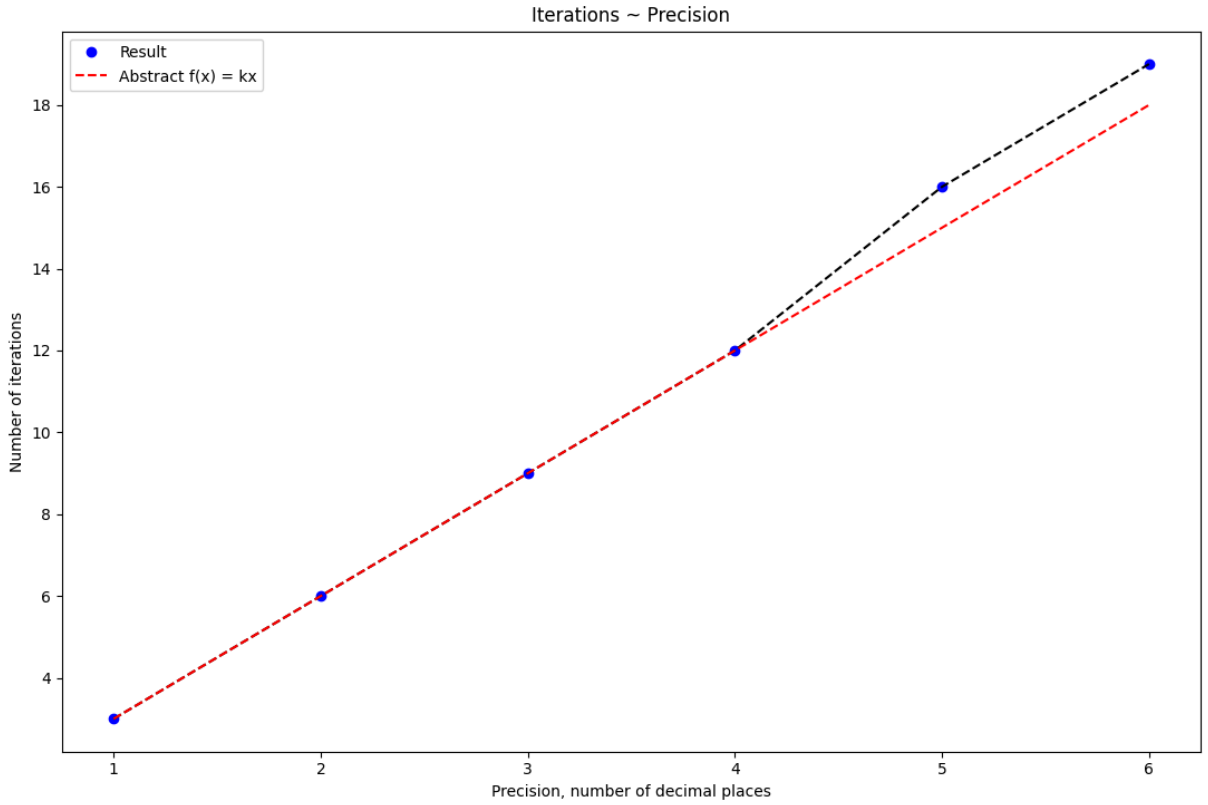

Рисунок 2. График зависимости количества итераций от порядка точности

Если провести исследование графика, можно понять, что количество итераций растет линейно относительно порядка точности ε (количества знаков после запятой).

При непосредственная зависимость числа итераций от точности ε будет логарифмической (рисунок 3): с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

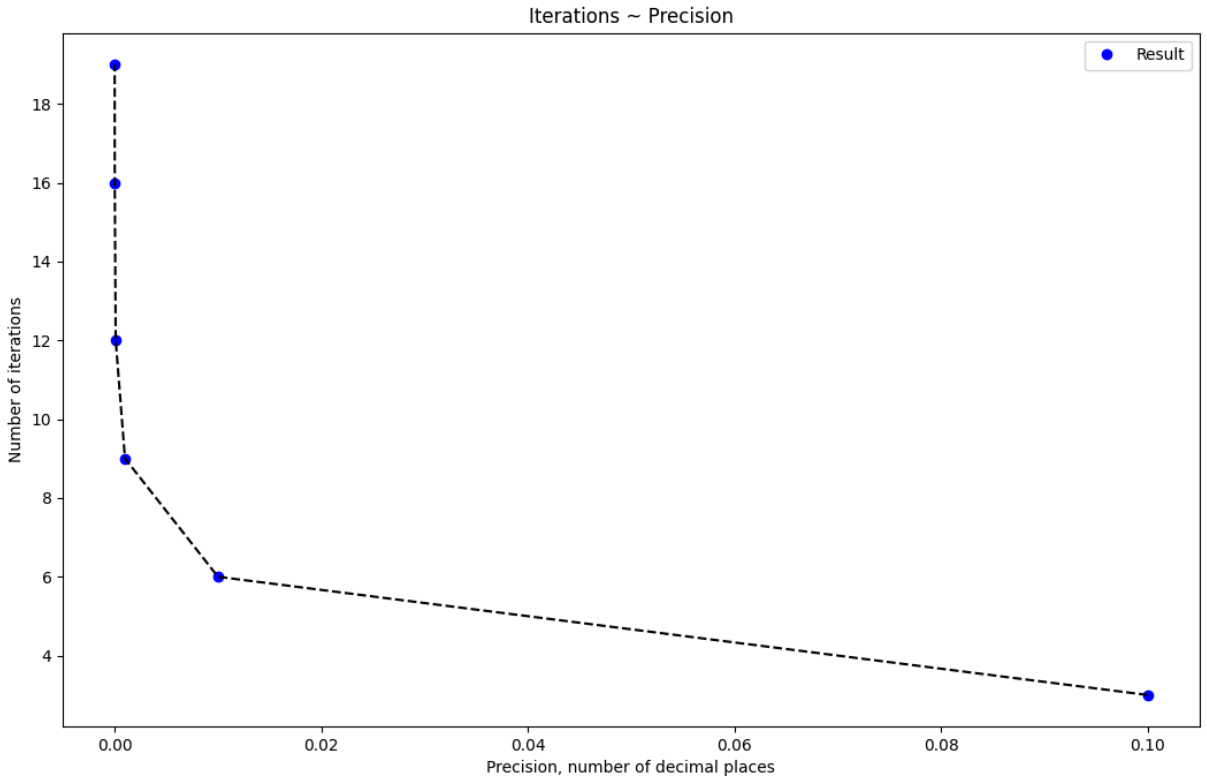

Рисунок 3. График зависимости количества итераций от точности

5) Для исследования чувствительности метода ко входным данным была написана дополнительная программа. Программа производит итерацию точности delta (от 0.1 до 10^-6) и применяет алгоритм из пункта 3. Результаты работы метода бисекции при различных точностях delta представлены в ниже:

```
For Delta = 0.100000:
     For epsilon = 0.100000: x = 0.900000, iter = 3
     For epsilon = 0.010000: x = 1.000000, iter = 6
     For epsilon = 0.001000: x = 1.000000, iter = 9
     For epsilon = 0.000100: x = 1.000000, iter = 12
     For epsilon = 0.000010: x = 1.000000, iter = 16
     For epsilon = 0.000001: x = 1.000000, iter = 19
```

```
For Delta = 0.010000:
     For epsilon = 0.100000: x = 0.940000, iter = 3
     For epsilon = 0.010000: x = 1.010000, iter = 6
     For epsilon = 0.001000: x = 1.000000, iter = 9
     For epsilon = 0.000100: x = 1.000000, iter = 12
     For epsilon = 0.000010: x = 1.000000, iter = 16
     For epsilon = 0.000001: x = 1.000000, iter = 19
For Delta = 0.001000:
     For epsilon = 0.100000: x = 0.943000, iter = 3
     For epsilon = 0.010000: x = 1.006000, iter = 6
     For epsilon = 0.001000: x = 1.001000, iter = 9
     For epsilon = 0.000100: x = 1.000000, iter = 12
     For epsilon = 0.000010: x = 1.000000, iter = 16
     For epsilon = 0.000001: x = 1.000000, iter = 19
For Delta = 0.000100:
     For epsilon = 0.100000: x = 0.943200, iter = 3
     For epsilon = 0.010000: x = 1.005700, iter = 6
     For epsilon = 0.001000: x = 1.001000, iter = 9
     For epsilon = 0.000100: x = 1.000100, iter = 12
     For epsilon = 0.000010: x = 1.000000, iter = 16
     For epsilon = 0.000001: x = 1.000000, iter = 19
For Delta = 0.000010:
     For epsilon = 0.100000: x = 0.943210, iter = 3
     For epsilon = 0.010000: x = 1.005720, iter = 6
     For epsilon = 0.001000: x = 1.001030, iter = 9
     For epsilon = 0.000100: x = 1.000050, iter = 12
     For epsilon = 0.000010: x = 0.999990, iter = 16
     For epsilon = 0.000001: x = 1.000000, iter = 19
For Delta = 0.000001:
     For epsilon = 0.100000: x = 0.943213, iter = 3
     For epsilon = 0.010000: x = 1.005715, iter = 6
     For epsilon = 0.001000: x = 1.001028, iter = 9
     For epsilon = 0.000100: x = 1.000051, iter = 12
     For epsilon = 0.000010: x = 0.999990, iter = 16
     For epsilon = 0.000001: x = 1.000001, iter = 19
```

Исходя из результатов исследования, можно сделать вывод об устойчивости метода отрезков к ошибкам в округлении.

Выводы.

В ходе работы был изучен метод бисекции (половинного деления). Для уравнения $f(x) = 2x^2 - x^4 - 1 + ln(x)$ корень был успешно найден на отрезке [0.6432; 1.4432341923] с точностью от 0.1 до 10^-6. Построенный график показал линейную зависимость числа итераций от порядка точности, а также логарифмическую зависимость числа итераций от точности: увеличение

точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод бисекции является устойчивым к малым ошибкам округления значений функции, однако при грубых возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `precision.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

// using namespace std;


double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

int main(int argc, char* argv[]) {
    int n;
    double x;
    double eps;
    double delta = pow(10, -4);

    FILE *out_file = fopen(argv[1], "w");

    double a = 0.13453;
    double b = 1.4432341923;

    for (int i=1; i<7; i++) {
        eps = pow(10, -i);
        x = BISECT(a, b, eps, n);

         printf("\tFor epsilon = %lf: x = %lf, iter = %d\n", eps,
Round(x, delta), n);

        fprintf(out_file, "%d %d\n", i, n);
    }

    fclose(out_file);

    return 0;
}
```

Файл `input_errors.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>
```

```
#include "methods.h"

double F(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

int main() {
    int n;
    double x;
    double eps;
    double delta;

    double a = 0.13453;
    double b = 1.4432341923;

    for (int di=1; di<7; di++) {
        delta = pow(10, -di);
        printf("For Delta = %lf:\n", delta);

        for (int i=1; i<7; i++) {
            eps = pow(10, -i);
            x = BISECT(a, b, eps, n);

                printf("\tFor epsilon = %lf: x = %lf, iter = %d\n",
eps, Round(x, delta), n);
        }
    }

    return 0;
}
```

Файл `graph.py`:

```
import matplotlib
matplotlib.use("TkAgg")

import os
import sys
from matplotlib import pyplot as plt

data_x = []
data_y = []

if not os.path.exists(sys.argv[1]):
    raise FileNotFoundError(f"No {sys.argv[1]} file!")
file = open(sys.argv[1], "r")

for line in file.readlines():
    if line:
        x, y = line.split()
        data_x.append(int(x))
        data_y.append(int(y))

plt.plot(data_x, data_y, "bo", label="Result")
plt.plot(data_x, data_y, "k--")
plt.plot(data_x,   list(map(lambda   x:   3*x,   data_x)),   "r--",
label="Abstract f(x) = kx")
```

```
plt.title("Iterations ~ Precision")
plt.xlabel("Precision, number of decimal places")
plt.ylabel("Number of iterations")
plt.legend()
plt.show()
```

Файл `Makefile`:

```
SRC_DIR = ${CURDIR}/src
WORK_DIR = ${SRC_DIR}/workdir
MAT_DIR = ${SRC_DIR}/Materials


all: ${WORK_DIR}/precision.txt ${WORK_DIR}/graph.py
python3 ${WORK_DIR}/graph.py ${WORK_DIR}/precision.txt

input_errors: ${WORK_DIR}/input_errors.o
${WORK_DIR}/input_errors.o

${WORK_DIR}/precision.txt: ${WORK_DIR}/precision.o
${WORK_DIR}/precision.o ${WORK_DIR}/precision.txt

${WORK_DIR}/precision.o: ${WORK_DIR}/precision.cpp ${MAT_DIR}
g++     -g     -I${MAT_DIR}     ${WORK_DIR}/precision.cpp     -o
${WORK_DIR}/precision.o

${WORK_DIR}/input_errors.o:           ${WORK_DIR}/input_errors.cpp
${MAT_DIR}
g++    -g    -I${MAT_DIR}    ${WORK_DIR}/input_errors.cpp    -o
${WORK_DIR}/input_errors.o

help:
@echo "Available options:"
@echo "    make [all] -- calculate f(x) = 0 and show results"
@echo "    make input_errors -- show input data sensitivity"
```